

Điều khiển UR3e viết chữ trên mặt phẳng 2D bằng ROS 2 Humble, MoveIt 2 và Ignition Gazebo

Sinh viên: Việt Anh · MSSV: · Lớp:

Mục	Nội dung
Robot	Universal Robots UR3e, mô phỏng trong Ignition Gazebo Fortress
Nền tảng	Ubuntu 22.04 · ROS 2 Humble · MoveIt 2 · ros2_control
Bài toán	Vẽ một đường tròn, chữ V và chữ A trên cùng một mặt phẳng, các hình cách nhau không vẽ chồng lên nhau
Repo GitHub	https://github.com/vanhhhh512/ur3e_va
Video demo	(điền link Google Drive, chế độ Anyone with the link)

1. Cấu trúc chương trình

Chương trình được tách làm ba lớp rõ ràng. Lớp hình học chỉ làm việc với toạ độ 2D trên mặt phẳng giấy và hoàn toàn không biết gì về robot. Lớp ánh xạ đổi toạ độ 2D đó thành pose 6 chiều của khâu tác động cuối. Lớp điều khiển đẩy chuỗi pose sang MoveIt 2 để lập kế hoạch rồi thực thi. Nhờ tách như vậy, muốn đổi chữ hay đổi kích thước bản vẽ thì chỉ sửa một file YAML, không phải chạm vào mã C++ điều khiển.

Luồng dữ liệu khi chạy `demo_all.launch.py`:

```

canvas.yaml (tham số hình học)
↓
strokes.cpp → danh sách waypoint 2D (u, v), lấy mẫu 2 mm
↓
layout.cpp → kiểm tra khoảng hở giữa các hình và tâm với (dừng nếu sai)
↓
canvas.cpp → đổi (u, v) thành geometry_msgs/Pose của tool0
↓
MoveGroupInterface (MoveIt 2) → lập kế hoạch, sinh joint trajectory
↓
joint_trajectory_controller → UR3e trong Ignition Gazebo
↓
ink_trail.cpp → đọc TF đầu bút, vẽ nét mực trong RViz và ghi CSV

```

2. Các node và package chính

Toàn bộ mã do sinh viên viết nằm trong một package duy nhất là `ur_pen_plotter`. Các package mô phỏng UR (`ur_simulation_gz`, `ur_moveit_config`, `ur_description`) dùng bản cài sẵn từ apt, không chép vào repo.

Thành phần	Loại	Nhiệm vụ
<code>draw_shapes_node</code>	Node C++	Node điều khiển chính: dựng waypoint, gửi MoveIt, lập kế hoạch, thực thi, ghi lại vệt bút
<code>stroke_preview</code>	Node C++	Kiểm tra bố trí và xuất quỹ đạo dự định ra CSV, chạy được khi chưa bật Gazebo

demo_all.launch.py	Launch	Chạy trọn bộ bằng một lệnh: Gazebo, ros2_control, MoveIt, RViz và node điều khiển
bringup_sim.launch.py	Launch	Chỉ dựng môi trường mô phỏng
draw.launch.py	Launch	Chỉ chạy node điều khiển khi môi trường đã sẵn sàng
canvas.yaml	Cấu hình	Toàn bộ hình học bản vẽ và tham số thực thi
ur3e_pen.urdf.xacro	Mô hình	UR3e gắn thêm một cây bút dài 10 cm vào mặt bích tool0

3. Cách tạo quỹ đạo

3.1. Hệ toạ độ mặt phẳng vẽ

Mặt phẳng vẽ dựng đứng, vuông góc với trục X của hệ world và cách đế robot một khoảng $plane_x = 0,40$ m. Trên mặt phẳng dùng một hệ hai chiều riêng: trục u chạy theo chiều $-Y$ của world, trục v chạy theo $+Z$. Một điểm (u, v) được đổi thành pose của tool0 theo hai bước:

- Hướng bút giữ cố định, trục Z của tool0 chĩa theo $+X$ của world, tức bút vuông góc với mặt phẳng giấy — tương ứng quaternion $w = y = \sqrt{0,5}$.
- Vị trí lùi lại đúng chiều dài bút, vì vật phải chạm mặt phẳng là đầu bút chứ không phải mặt bích. Cây bút là một link thật trong URDF nên hình ảnh trong Gazebo và phép tính trùng nhau.

3.2. Sinh waypoint cho từng hình

Mỗi hình gồm một hoặc nhiều nét (stroke) — một đường liền mà bút không rời mặt phẳng. Các điểm được lấy mẫu cách nhau 2 mm dọc theo nét:

- Đường tròn: quét góc từ đỉnh trên theo chiều kim đồng hồ trọn 2π , điểm cuối trùng điểm đầu nên nét khép kín. Số điểm tự tính theo chu vi chia cho bước lấy mẫu.
- Chữ V: hai đoạn thẳng nối trên-trái → đỉnh nhọn dưới → trên-phải, đi liền một nét.
- Chữ A: nét thứ nhất là khung nhọn dưới-trái → đỉnh → dưới-phải; nét thứ hai là thanh ngang. Ở độ cao bằng 40 % chiều cao tính từ đáy, hai cạnh xiên thu hẹp còn 60 % nửa bề rộng, nên thanh ngang được cắt đúng bằng khoảng đó để hai đầu chạm sát hai cạnh.

Giữa hai nét robot nhấc bút lùi ra 5 cm, trượt ngang tới vị trí nét mới rồi hạ xuống — đúng yêu cầu của đề về việc chia chữ thành nhiều nét.

3.3. Kiểm tra bố trí trước khi chạy

Trước khi gửi bất kỳ lệnh nào xuống robot, hàm `checkLayout()` duyệt mọi cặp nét thuộc hai hình khác nhau, tính khoảng cách nhỏ nhất giữa hai tập điểm và so với ngưỡng 30 mm; đồng thời tính tầm với lớn nhất của tool0 đo từ trục khớp vai rồi so với ngưỡng 0,45 m (UR3e với tối đa 0,50 m). Sai một trong hai thì chương trình dừng hẳn. Hai nét của chữ A được phép cắt nhau vì chúng được đánh dấu cùng thuộc một chữ.

4. MoveIt lập kế hoạch và gửi trajectory tới robot

Node điều khiển làm việc với MoveIt 2 qua lớp `MoveGroupInterface` trên nhóm khớp `ur_manipulator`, dùng hai kiểu lập kế hoạch cho hai mục đích khác nhau:

Tình huống	Cách lập kế hoạch	Lý do
------------	-------------------	-------

Thoát điểm kỳ dị, tiếp cận điểm chờ đầu tiên, đổ tay sau khi vẽ xong	<code>setJointValueTarget / setPoseTarget</code> rồi <code>plan()</code> và <code>execute()</code> — bộ lập kế hoạch OMPL	Đây là các chuyển động tự do trong không gian, hướng của tool thay đổi nhiều, không cần đi theo đường thẳng
Hạ bút, vẽ nét, nhắc bút, trượt ngang giữa hai nét	<code>computeCartesianPath(waypoints, 0.005, 0.0, trajectory, true)</code> rồi <code>execute()</code>	Nét vẽ bắt buộc phải bám đúng hình, nên nội suy thẳng trong không gian Cartesian với bước 5 mm và có kiểm tra va chạm

Kết quả của cả hai đường đều là một `moveit_msgs/RobotTrajectory`. MoveIt gửi quỹ đạo này xuống `joint_trajectory_controller` qua `action /joint_trajectory_controller/follow_joint_trajectory`; controller chạy trong Gazebo thông qua plugin `ign_ros2_control` gắn trong URDF. Node chỉ bắt đầu vẽ sau khi action server đã sẵn sàng.

4.1. Hai vấn đề gặp phải và cách xử lý

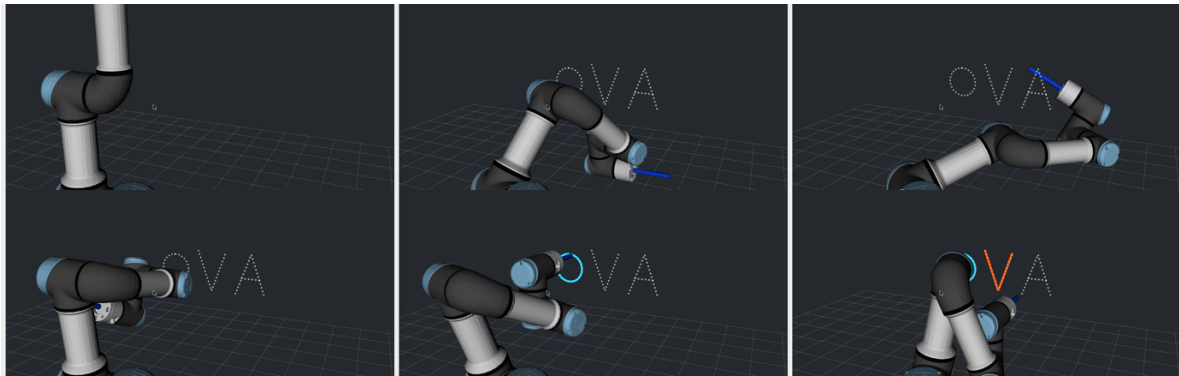
Khi robot bắt đầu trong Gazebo, toàn bộ khớp bằng 0 nên cánh tay duỗi thẳng — đúng một điểm kỳ dị, ma trận Jacobian suy biến và mọi lời gọi IK đều hỏng. Vì vậy việc đầu tiên node làm luôn là đưa robot về tư thế gập $q_{ready} = [0, -\pi/2, \pi/2, -\pi/2, -\pi/2, 0]$.

Vấn đề thứ hai đến từ việc MoveIt chọn nghiệm động học ngược một cách ngẫu nhiên. Có nghiệm khiến đường hạ bút thẳng đứng cắt qua trạng thái tự va chạm, khi đó `computeCartesianPath` chỉ đi được một phần đường. Node xử lý bằng cách quay về tư thế ready để MoveIt bốc nghiệm khác rồi tiếp cận lại, tối đa ba lần; nếu vẫn không được thì bỏ ràng buộc hạ thẳng và để OMPL tự tìm đường tới điểm chạm. Tương tự, mỗi lần gọi `plan()` được thử lại tối đa năm lần vì OMPL lấy mẫu ngẫu nhiên nên đôi khi trả về đường bị MoveIt từ chối.

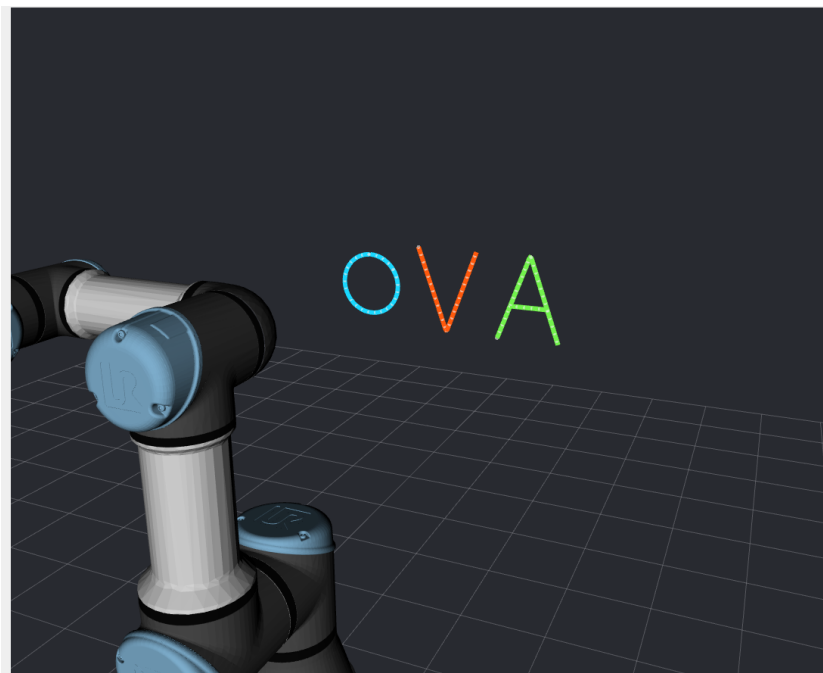
5. Kết quả chạy thử

Vị trí thực tế của đầu bút được lấy từ TF sau mỗi 25 ms, vẽ thành nét mực trong RViz và ghi song song ra CSV. Nhờ đó sai lệch giữa quỹ đạo dự định và quỹ đạo robot đi thật được đo bằng số. Bảng dưới là kết quả một lần chạy đầy đủ với tham số mặc định:

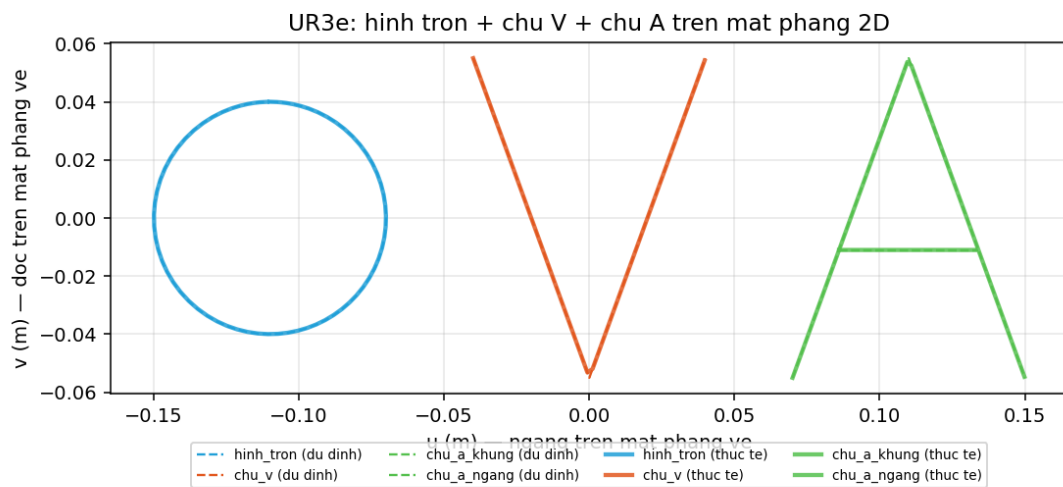
Chỉ số	Giá trị đo được
Khoảng hở nhỏ nhất giữa các hình	44,6 mm (tròn ↔ V); các cặp còn lại 66–156 mm
Tầm với lớn nhất của tool0	0,359 m so với giới hạn đặt 0,45 m
Chiều dài nét vẽ	tròn 25,1 cm · V 23,4 cm · khung A 23,4 cm · ngang A 4,8 cm
Tỉ lệ đường Cartesian tính được	100 % ở tất cả các đoạn
Sai số bám quỹ đạo, trung bình	0,46 – 0,52 mm tùy nét
Sai số bám quỹ đạo, lớn nhất	0,99 mm
Độ lệch đầu bút khỏi mặt phẳng vẽ	lớn nhất 0,07 mm
Kiểm thử đơn vị	8/8 test hình học đạt



Hình 1. Sáu thời điểm trong một lần chạy. Chuỗi điểm mờ là waypoint dự định, robot vẽ đè nét mực lên trên.



Hình 2. Kết quả cuối trong RViz. Robot đã đổ sang bên để không che bản vẽ.



Hình 3. Đối chiếu quỹ đạo dự định (nét đứt) với vết bút thực tế (nét liền) trong hệ tọa độ mặt phẳng vẽ. Hai đường gần như trùng khít.

Cách chạy lại

```
colcon build --symlink-install
source install/setup.bash
ros2 launch ur_pen_plotter demo_all.launch.py wait_for_button:=true
```

Với tham số `wait_for_button:=true`, robot đứng yên cho tới khi bấm nút Next trong panel RvizVisualToolsGui của RViz — tiện khi cần quay video. Hướng dẫn cài đặt và chạy đầy đủ nằm trong README của repo.

Liên kết nội bài

Mục	Đường dẫn
Mã nguồn GitHub (public)	https://github.com/vanhhhh512/ur3e_va
Video demo	(điền link Google Drive)